2.10.1 Please Welcome Profiling

And so, we are stuck with *profiling*. If you are asking yourself "what's profiling?" or "where do I start?", here there are a few tips:

1. Profiling means measuring the running time and possibly the memory consumption of different parts of your code.
2. It can be done at a high level (measuring the calls to high level functions) or lower level, for each instruction, and although you can set it up manually (measuring the time before and after a call), there are great tools to aid you – usually guaranteeing an error-free measurement.
3. Profiling can't give you general-purpose answers: it can measure the performance of *your* code on the input you provide.
    a) In turn, this means that the result of profiling is as good as the input you use, meaning that if you only use a very specific input, you might tune your code for an edge case, and it could perform poorly on different inputs. Also, another key factor is the data volume: to be statistically significant, it can be nice to collect profiling results on many runs over (pseudo)random inputs.
    b) It also means that results are not generalizable to different programming languages, or even different implementations in the same language.
4. Profiling requires time. The more in depth you go with tuning, the more time it requires. Remember Donald